Instruction Following in Small Models: Qwen2.5 on IFEval

Module 3: Evaluation · October 2026

Summary

We built a re-runnable harness for the IFEval benchmark and used it to measure Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct on the specified hardware, an NVIDIA V100. The 0.5B model fully follows 26.1% of prompts (prompt-level strict, 95% CI 22.5–29.9%), close to the 27.9% Qwen reports for it. The 1.5B model reaches 38.3%. The IFEval paper reports 43.1% for PaLM 2 S and 76.9% for GPT-4.

Repeated runs gave byte-identical outputs, and the official scorer reproduces the paper's GPT-4 strict scores to within 0.2 points, so the gap to the paper is not a harness artefact. Going from 0.5B to 1.5B alone adds 12.2 points, while every other confound we measured is worth a few points at most. We read the gap as mainly a matter of model scale.

Prompt-level strict	Score
Qwen2.5-0.5B (ours, V100)	26.1% (22.5–29.9)
Qwen2.5-1.5B (ours, V100)	38.3% (34.3–42.4)
PaLM 2 S (IFEval paper)	43.1%
GPT-4 (IFEval paper)	76.9%

1. Setup

IFEval has 541 prompts containing 834 instructions that can be checked by code, such as "answer in all lowercase" or "use no commas". The paper evaluated GPT-4 and PaLM 2 S, not Qwen, so this is not a reproduction of a paper number. We compare our models against those numbers and against Qwen's own report.

The pipeline renders each prompt with the model's chat template, generates one response, scores it with the official Google scorer (unmodified), and computes the four standard metrics with Wilson 95% intervals. Each run writes its own directory with the exact prompts, raw outputs, scorer output and a manifest of every setting actually used.

Setting	Value
Models	Qwen2.5-0.5B-Instruct `7ae5576`, Qwen2.5-1.5B-Instruct `989aa79`
Hardware	Tesla V100-SXM2-32GB (department DGX, GPU 1)
Precision, attention	fp16, SDPA
Decoding	greedy, max 1024 new tokens, batch size 8
Scorer	official IFEval scorer, commit `ff4d1a6`
Software	torch 2.4.1 (CUDA 11.8 build), transformers 4.44.2, Python 3.10.14

Two settings are easy to miss. With an empty system prompt, Qwen's chat template inserts its own ("You are Qwen, created by Alibaba Cloud. You are a helpful assistant."). And the models' own generation config adds `repetition_penalty = 1.1`, which still applies under greedy decoding. We kept both as shipped.

---

2. Checking the scorer

The scorer repository includes the GPT-4 responses used in the paper. Scoring them with our pinned scorer seven times gave prompt-level strict 76.89–77.08 and instruction-level strict 83.57–83.69, against 76.89 and 83.57 in the paper's Table 3 (an exact match in 6 of 7 scorings). The loose scores came out 0.2–0.4 points above the paper's every time. We could not explain that residual, but it is too small to matter here.

We found two problems in the official benchmark. We documented both and did not patch either.

1. **The scorer is not fully deterministic.** Two prompts (keys 1122 and 1129) ask about the characters `#` and `!`. The letter-frequency checker rejects them and substitutes a random letter on every run. Scoring identical text therefore moves instruction-level accuracy by up to 0.24 points.
2. **The benchmark changed after publication.** Prompt 2785 now asks for "at least 3 placeholders", but the paper's GPT-4 answer was written for "at least one". We remapped that one response to score the GPT-4 file.

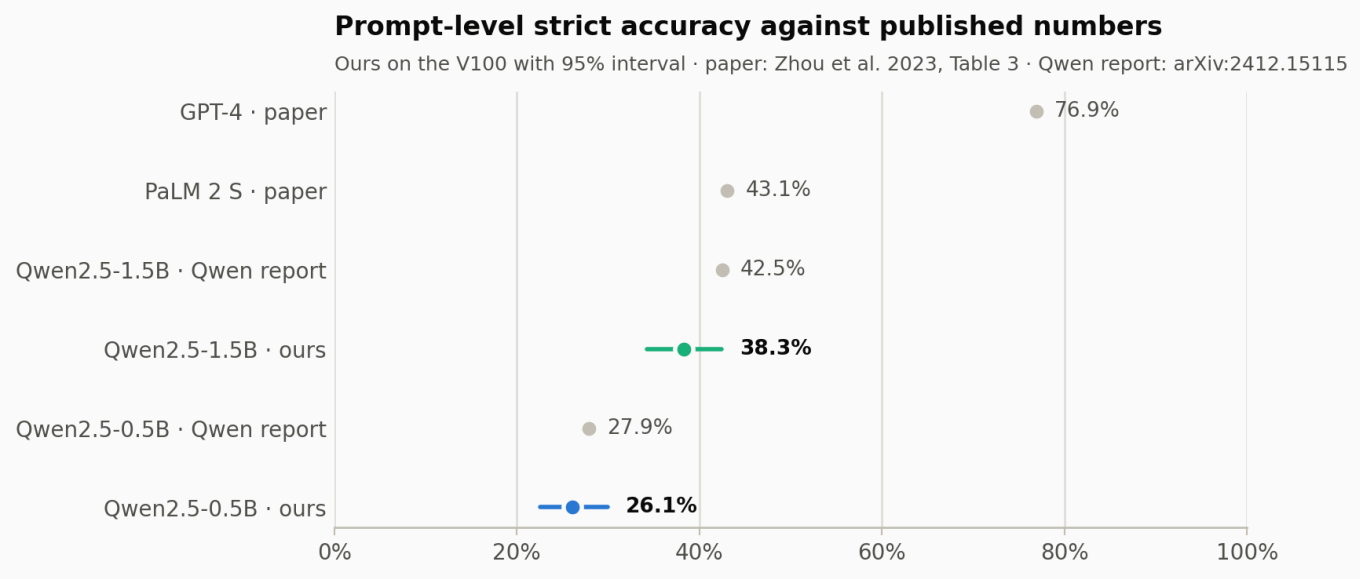
---

3. Results

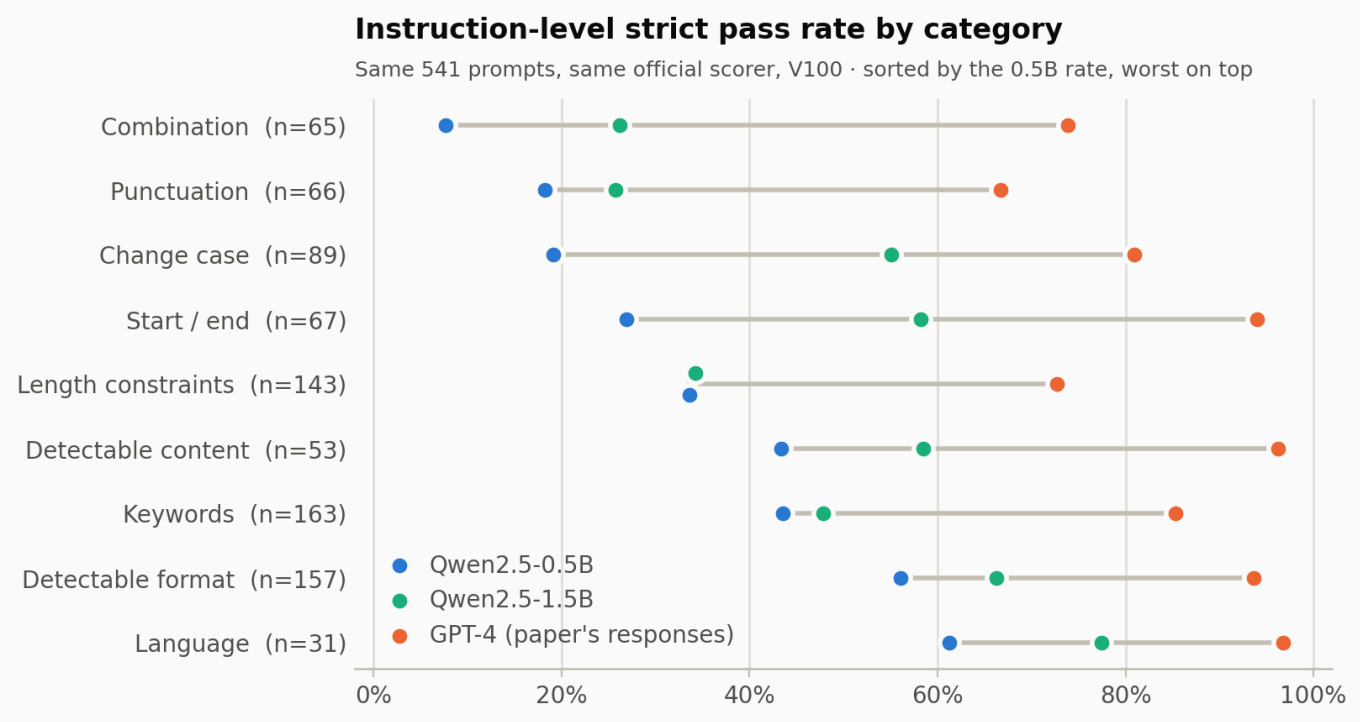
Metric	Qwen2.5-0.5B	Qwen2.5-1.5B
Prompt-level strict	26.1% (22.5–29.9)	38.3% (34.3–42.4)
Instruction-level strict	36.1% (32.9–39.4)	48.9% (45.5–52.3)
Prompt-level loose	28.1% (24.5–32.0)	42.9% (38.8–47.1)
Instruction-level loose	38.1% (34.9–41.5)	53.5% (50.1–56.8)

Repeatability. We ran each model three times with identical settings. All three runs produced byte-identical outputs. Prompt-level strict across the repeats was 26.06% [26.06–26.06] for 0.5B, a spread of 0.0 points, and 38.14% [38.08–38.26] for 1.5B. The small 1.5B spread comes from the scorer's random letter check, since the text being scored was identical.

Comparison with published numbers. For 0.5B, Qwen's reported 27.9% lies inside our interval. For 1.5B, Qwen reports 42.5%, which is just above our strict interval but inside our loose one. Qwen does not say which metric it reports.



Scale. Moving from 0.5B to 1.5B adds 12.2 points prompt-level strict. Paired by prompt, 105 prompts fail with 0.5B and pass with 1.5B, and 39 go the other way (exact McNemar test, $p < 0.001$). The gains are uneven: change-case instructions rise from 19% to 55% and start/end instructions from 27% to 58%, while length constraints stay at 34%.

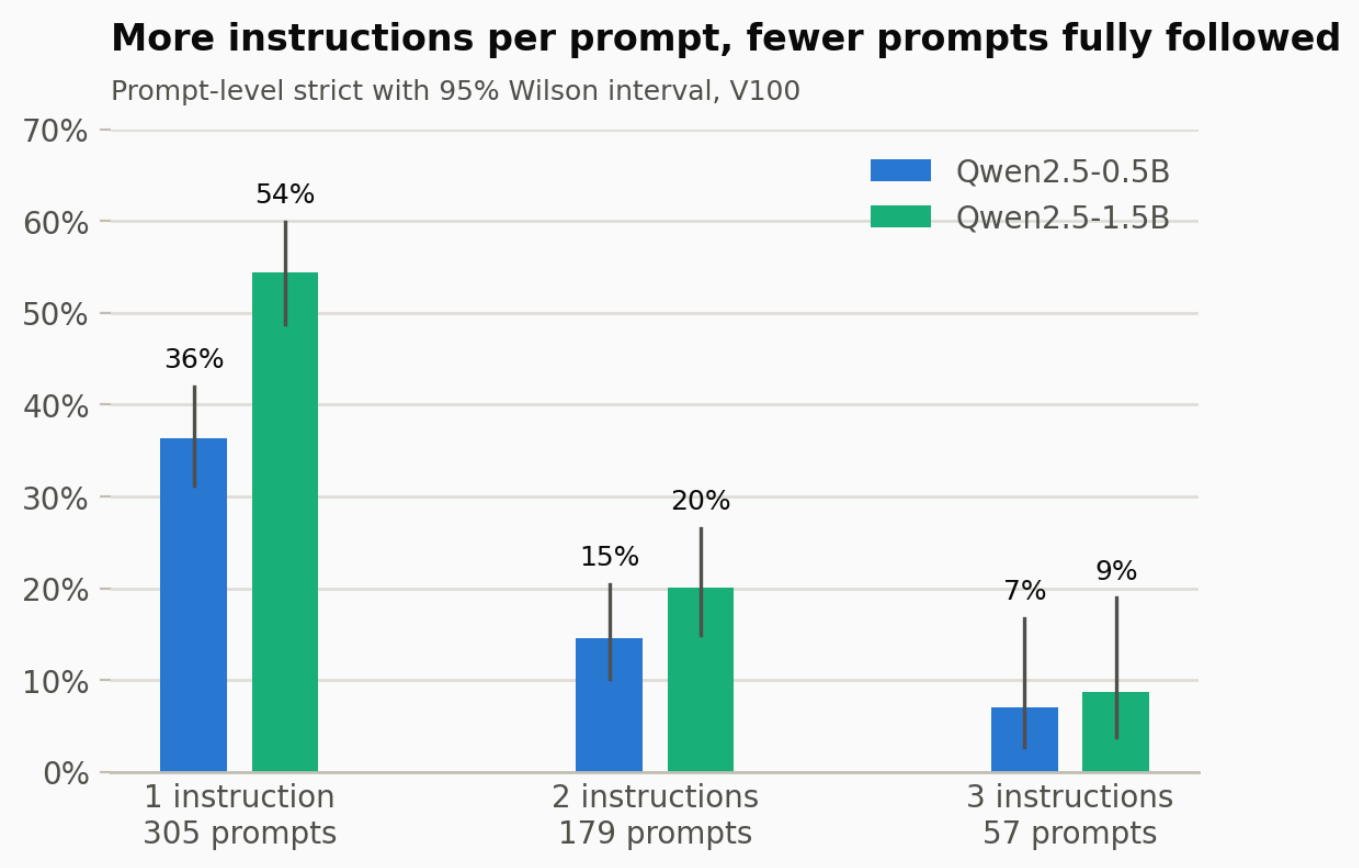


Where the models fail. The models usually complete the task but ignore the formatting rule attached to it. Typical 0.5B failures:

Instruction	Pass rate	What the model did
Separate paragraphs with `***`	0 / 27	used blank lines; only 4 responses contained `***`
Repeat the request, then answer	3 / 41	answered without repeating
Two responses separated by `******`	2 / 24	wrote a numbered list instead
Write in all lowercase	6 / 39	used normal capitalisation ("Dear [Friend's Name],")

Instruction	Pass rate	What the model did
Use no commas	12 / 66	used commas from the first line

Rules that match normal writing (a title, JSON, a postscript) pass far more often. Accuracy also falls sharply as instructions stack up. The 0.5B model fully follows 36.4% of prompts with one instruction, 14.5% with two and 7.0% with three (1.5B: 54.4%, 20.1%, 8.8%).



Truncation. 20 of 541 responses (3.7%) hit the 1024-token limit, and 9 of those still passed. Even if the other 11 had passed, the score would rise by at most 2.0 points.

4. Sensitivity checks

System prompt. We replaced Qwen's default system prompt with three alternatives on the 0.5B model, changing nothing else:

System prompt	Prompt-level strict	Prompts gained / lost	p
Qwen default (baseline)	26.1%		
"You are a helpful assistant."	29.8%	+49 / −29	0.03
"…Follow every instruction … exactly …"	27.0%	+36 / −31	0.63
"…warm, natural, conversational style."	26.2%	+38 / −37	1.00

Each variant changed the text of almost every response but moved the score by only +0.2 to +3.7 points. The generic prompt has p = 0.03, which does not survive a Bonferroni correction for three

tests (threshold 0.017). It pointed the same way on the laptop (+2.0 points, p = 0.25), so the effect may be real but small. Explicitly asking the model to follow every instruction did not help.

Hardware. Before the V100 was available we ran the same four configurations on a 4 GB laptop GPU (RTX 3050 Ti, batch 4, CUDA 12.1):

Configuration	Laptop	V100	Change	p
Qwen default (baseline)	26.6%	26.1%	−0.55	0.38
"You are a helpful assistant."	28.7%	29.8%	+1.11	0.24
"...Follow every instruction ..."	26.4%	27.0%	+0.55	0.45
"...conversational style."	27.0%	26.2%	−0.74	0.29

About 36% of responses differ word for word between the two machines, but no score moved by more than 1.1 points and none of the changes is significant. The GPU, batch size and CUDA version all changed together, so this measures their combined effect only.

---

5. Reading the gap

Against Qwen's report. For 0.5B the gap is 1.8 points, well inside our interval of ±3.7 points, so our harness agrees with the model developer. For 1.5B the gap is 4.2 points. It is borderline on strict and inside the interval on loose, and since Qwen does not name its metric we cannot pin it down further.

Against PaLM 2 S (−17.0 points) and GPT-4 (−50.8). The scorer reproduces the paper's GPT-4 numbers, so the gap is not a measurement error. It is a difference between models of very different size. The confounds we could measure or bound:

Confound	Effect on our score
Model size, 0.5B → 1.5B	+12.2 points (measured)
System prompt	+0.2 to +3.7 points, not significant after correction
Truncation at 1024 tokens	at most +2.0 points
Hardware (laptop vs V100)	−0.7 to +1.1 points, not significant
Scorer randomness and benchmark edit	at most 0.4 points
Run-to-run variation	0 (identical outputs)
`repetition_penalty = 1.1`	not varied

Even added together, the confounds other than size come to less than 8 points, well short of the 17-point gap to PaLM 2 S. Tripling the model size within the same family already closes 12 of those 17 points. Together with the steep drop on multi-instruction prompts, this points to model capacity as the main explanation.

6. Limitations

- **Server adjustments.** The DGX's NVIDIA driver (450.119.03) supports only CUDA 11.0, so we used the CUDA 11.8 build of the same torch version. Its system Python 3.8 cannot load the official scorer, which needs Python 3.9+, so we used Python 3.10.14, as on the laptop. Both are recorded in every run's manifest.
- **Speed.** On the shared DGX, runs took 30–37 minutes each (5 hours 8 minutes for all nine), slower than the laptop's 17–19 minutes. We did not measure why; the shared server was heavily loaded when we logged in.
- **Statistics.** The intervals treat instructions as independent, although instructions in the same prompt are correlated, so instruction-level intervals are somewhat too narrow.
- **Not done.** We did not vary `repetition_penalty` or run the optional template ablation. Saving outputs after each batch would make interrupted runs resumable, but we did not implement it.

7. Reproducing

On the DGX, inside `tmux`:

```
curl -LsSf https://astral.sh/uv/install.sh | sh && export PATH="$HOME/.local/bin:$PATH"
git clone https://github.com/runjanna26/module_3_evaluation.git && cd module_3_evaluation
uv venv --python 3.10.14 --seed .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/pip uninstall -y torch     # only on a CUDA 11 driver, as on this DGX
.venv/bin/pip install torch==2.4.1 --index-url https://download.pytorch.org/whl/cu118
./scripts/setup.sh
./scripts/run_schedule.sh 1          # 1 = booked GPU; all 9 runs, about 5 hours
```

`src/aggregate_runs.py` reports the spread across run directories, `report/evaluation_figures.ipynb` regenerates every figure from `runs/`, and `scripts/make_pdf.py` renders this report.

Run directories in `runs/`	Contents
`qwen05b_hf_*_IST-DGX02-gpu1-{1,2,3}`	0.5B, V100, three repeats (run 1 is the baseline)
`qwen05b_sys_*_IST-DGX02-gpu1-{4,5,6}`	0.5B system-prompt variants, V100
`qwen15b_hf_*_IST-DGX02-gpu1-{7,8,9}`	1.5B, V100, three repeats
`qwen05b_laptop_*`	the same 0.5B configurations on the laptop
`qwen05b_hf_20260916_003724`	failed laptop attempt at batch 8 (out of memory), kept as a record

References

- Zhou et al. (2023). *Instruction-Following Evaluation for Large Language Models.* arXiv:2311.07911
- Qwen Team (2024). *Qwen2.5 Technical Report.* arXiv:2412.15115